

MediQueue

Project Report

Hospital Queue and Appointment Management System

Item	Details
Frontend	https://medi-queue-indol.vercel.app/
Backend API	https://mediqueue-8ofq.onrender.com
Stack	React, TypeScript, Vite, Node.js, Express, MongoDB, Mongoose
Testing Status	15 automated backend tests passing

This report documents the problem statement, solution approach, system design choices, object-oriented design, design patterns, SOLID principles, UML artifacts, database design, and verified test results for MediQueue.

1. Problem Statement

Hospitals often depend on manual appointment registers, walk-in queues, verbal status updates, and disconnected patient records. This causes long waiting times, unclear queue positions, scheduling conflicts, limited visibility for doctors, and operational overhead for administrators.

The problem addressed by MediQueue is to digitize and coordinate appointment booking, queue management, doctor consultation flow, and role-based hospital operations in a maintainable full-stack system.

2. Solution Approach

- Provide a React frontend with separate patient, doctor, and admin experiences.
- Use an Express and TypeScript backend to expose role-specific API routes.
- Use JWT authentication and role authorization to protect patient and doctor workflows.
- Persist hospital data in MongoDB through Mongoose models for users, appointments, queues, notifications, medical records, and settings.
- Represent core business rules with entities, interfaces, and design patterns so appointment creation, queue ordering, and notifications remain extensible.

3. Technology Stack

Layer	Technologies
Frontend	React 19, TypeScript, Vite, React Router, Axios, Framer Motion, Lucide React, React Hot Toast
Backend	Node.js, Express 5, TypeScript, tsx, nodemon
Database	MongoDB and Mongoose
Authentication	JWT and bcrypt
Development and Testing	npm, ESLint, TypeScript compiler, Node test runner, tsx
Deployment	Vercel frontend and Render backend

4. System Design Optimization

MediQueue applies system design principles by separating presentation, API routing, domain logic, persistence, and reusable behavioral modules. This separation improves scalability because each layer can evolve independently. The frontend can be deployed and cached separately from the backend, while the backend can be scaled as an API service behind Render.

Principle	How It Improves the System
Layered architecture	React handles UI, Express handles API orchestration, controllers handle use cases, models handle persistence, and patterns handle reusable domain behavior.
Role-based routing	Patient and doctor routes are protected by middleware, reducing accidental access and keeping authorization logic centralized.
Stateless API authentication	JWT-based requests avoid server-side session storage, making backend scaling easier.
Queue-specific data model	Daily queue documents group queue entries per doctor, making queue lookup and updates more focused than scanning all appointments.
Indexes in models	Appointment, queue, user, and medical record schemas define indexes for common lookup paths such as doctor/date, patient history, role/isActive, and queue entries.

Principle	How It Improves the System
Strategy-based queue ordering	Queue ordering can switch between FIFO, priority, and round-robin without rewriting queue manager logic.
Frontend API service layer	Axios services centralize HTTP calls and token injection, improving maintainability and reducing duplicated request code.

5. Architecture Explanation

The system follows a client-server architecture. The React frontend contains route-based pages and role dashboards. The Axios API layer sends requests to the Express backend. The backend validates users through authentication middleware, enforces role authorization, delegates business workflows to controllers, and stores persistent state through Mongoose models.

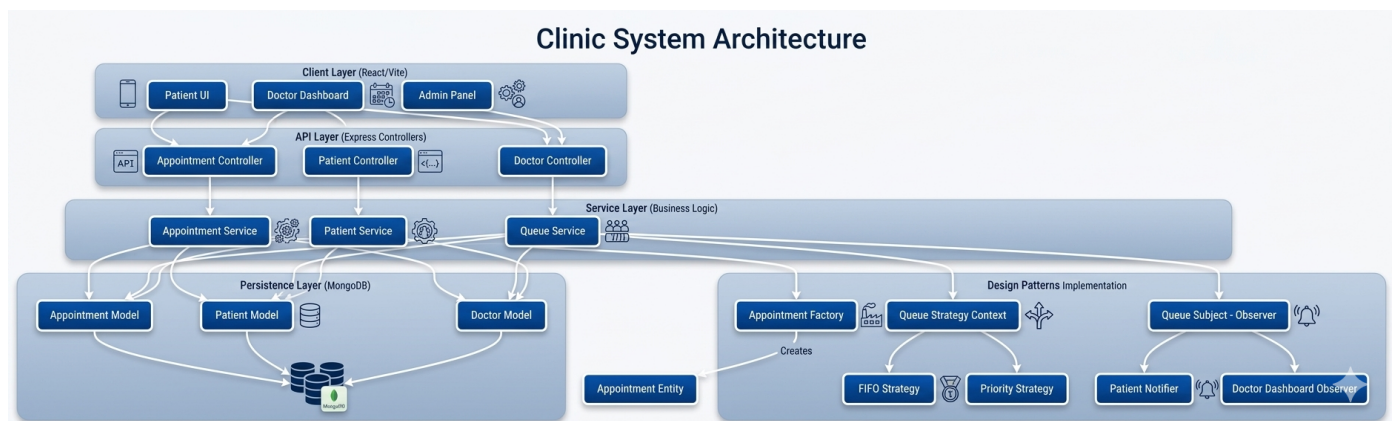


Figure 1. MediQueue system architecture.

6. OOP Concepts Used

OOP Concept	Where It Appears	Explanation
Encapsulation	Entities such as User, Patient, Doctor, Admin, and Appointment	State and behavior are grouped inside classes. Appointment methods such as confirm, cancel, and reschedule update internal appointment status consistently.
Inheritance	Patient, Doctor, and Admin extend User; appointment variants extend Appointment	Shared attributes are inherited from base classes, while specialized classes represent role-specific or appointment-type-specific behavior.
Polymorphism	Appointment factories and queue strategies	Different appointment factory classes expose the same create method; different queue strategies expose the same sort method but implement different ordering rules.
Abstraction	Interfaces in server/src/interfaces	Interfaces define contracts for users, doctors, patients, appointments, queue observers, queue subjects, queue strategies, and notification channels.

7. Design Patterns Implemented

Pattern	Implementation	Why It Was Used
Factory	appointment_factory.ts	Creates walk-in, scheduled, and emergency appointments without spreading type-checking logic across controllers or services.
Strategy	queue_strategy.ts	Allows queue ordering policies such as FIFO, priority, and round-robin to be changed independently from queue management logic.
Observer	queue_manager.ts and queue_observer.ts	Models notification of interested users when queue state changes.

Pattern	Implementation	Why It Was Used
Singleton	queue_registry.ts	Maintains one shared queue registry for doctor queues during application runtime.
Adapter	notification_adapter.ts	Normalizes email, SMS, and push notification providers behind one interface.
Composite	notification_composite.ts	Groups multiple notification channels so one notification event can fan out through several providers.

8. SOLID Principles

Principle	Reflection in Codebase
Single Responsibility	Controllers handle request workflows, models handle persistence schemas, middleware handles authentication and authorization, entities model domain behavior, and pattern files handle reusable algorithms.
Open/Closed	New appointment types can be added through a new factory; new queue ordering rules can be added by implementing the queue strategy interface without changing existing strategies.
Liskov Substitution	WalkInAppointment, ScheduledAppointment, and EmergencyAppointment can be used wherever Appointment is expected because they preserve the base appointment behavior.
Interface Segregation	Separate interfaces exist for user, patient, doctor, admin, appointment, queue strategy, queue observer, queue subject, and notification contracts.
Dependency Inversion	Queue behavior and notification behavior depend on interfaces/contracts rather than hard-coded concrete implementations, making strategies and adapters replaceable.

9. Major Workflow: Patient Appointment Booking

- Patient logs in and receives a JWT.
- Frontend stores the token and sends it with appointment booking requests.
- Authentication middleware verifies the token and role middleware confirms patient access.
- Patient controller creates an appointment with doctor, date, time, reason, type, and priority.
- If the appointment is for the current date, the controller creates or updates the doctor's queue and assigns a token number.
- MongoDB stores the appointment and queue entry, and the frontend displays updated appointment/queue information.

10. Database Design

Model	Purpose
UserModel	Stores patients, doctors, and admins with role-specific fields such as specialization, availability, managed doctors, and active status.
AppointmentModel	Stores appointment lifecycle, type, status, priority, time slot, cancellation, completion, and critical-case information.
QueueModel	Stores per-doctor daily queue state, queue entries, token number, current token, and queue-entry status.
NotificationModel	Stores user notification events and read/unread status.
MedicalRecordModel	Stores consultation outcome, diagnosis, notes, prescription, follow-up plan, and critical markers.
SystemSettingModel	Stores operational rules such as consultation duration, working hours, queue defaults, and appointment settings.

11. Test Cases and Results

The backend test suite uses Node's built-in test runner through tsx. The tests mock Mongoose model calls so controller, middleware, factory, and strategy behavior can be verified without starting Express or connecting to MongoDB.

Metric	Result
Command	cd server && npm test
Total tests	15
Passed	15
Failed	0

Area	Tested Behavior	Result
Appointment Factory	Correct factory selection and default priorities for scheduled/emergency appointments.	Pass
Authentication	Missing bearer tokens are rejected; valid JWTs attach authenticated users.	Pass
Authorization	Allowed roles continue; incorrect roles receive 403 forbidden response.	Pass
Patient Controller	Booking today's appointment, queue token assignment, cancellation, queue removal, and queue status calculation.	Pass
Doctor Controller	Doctor queue formatting and consultation completion with appointment, queue, and medical record updates.	Pass
Queue Strategies	FIFO, priority ordering, priority tie-breaking, and round-robin rotation.	Pass

12. Deployment and Runtime

Service	URL
Frontend	https://medi-queue-indol.vercel.app/
Backend API	https://mediqueue-8ofq.onrender.com
Local frontend	http://localhost:5173
Local backend	http://localhost:5000

13. Conclusion

MediQueue demonstrates a structured full-stack solution for hospital appointment and queue management. The project applies layered system design, object-oriented modeling, multiple design patterns, SOLID principles, MongoDB persistence, JWT-secured APIs, and automated backend tests. The architecture is prepared for further extension through additional admin APIs, integration tests, real notification providers, and more complete analytics dashboards.

Appendix A: UML and ER Diagrams

The following diagrams are included from the project repository.

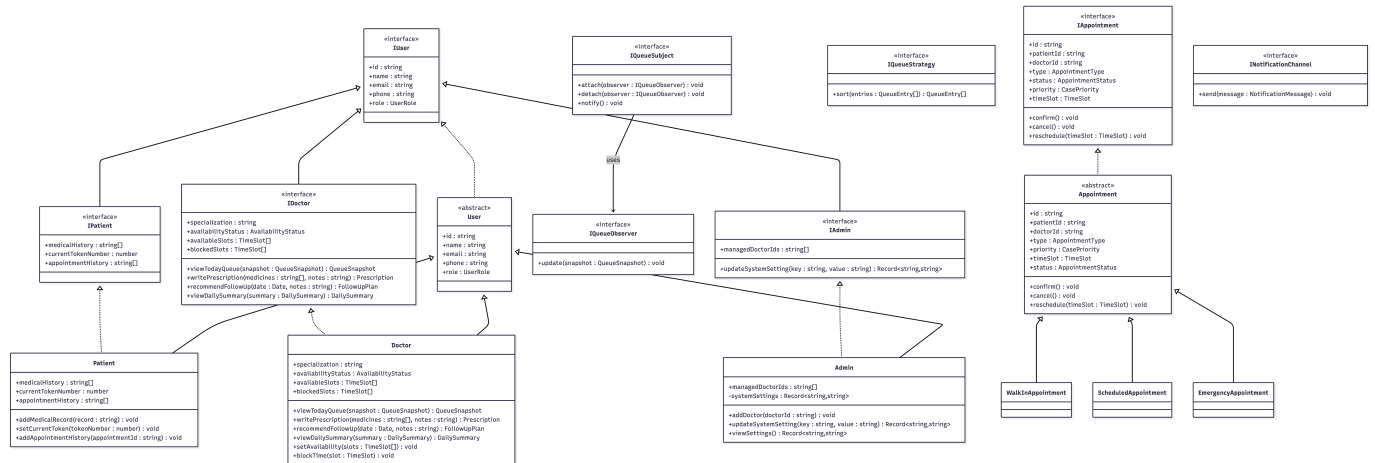


Figure 2. Class diagram showing the main entities and relationships.

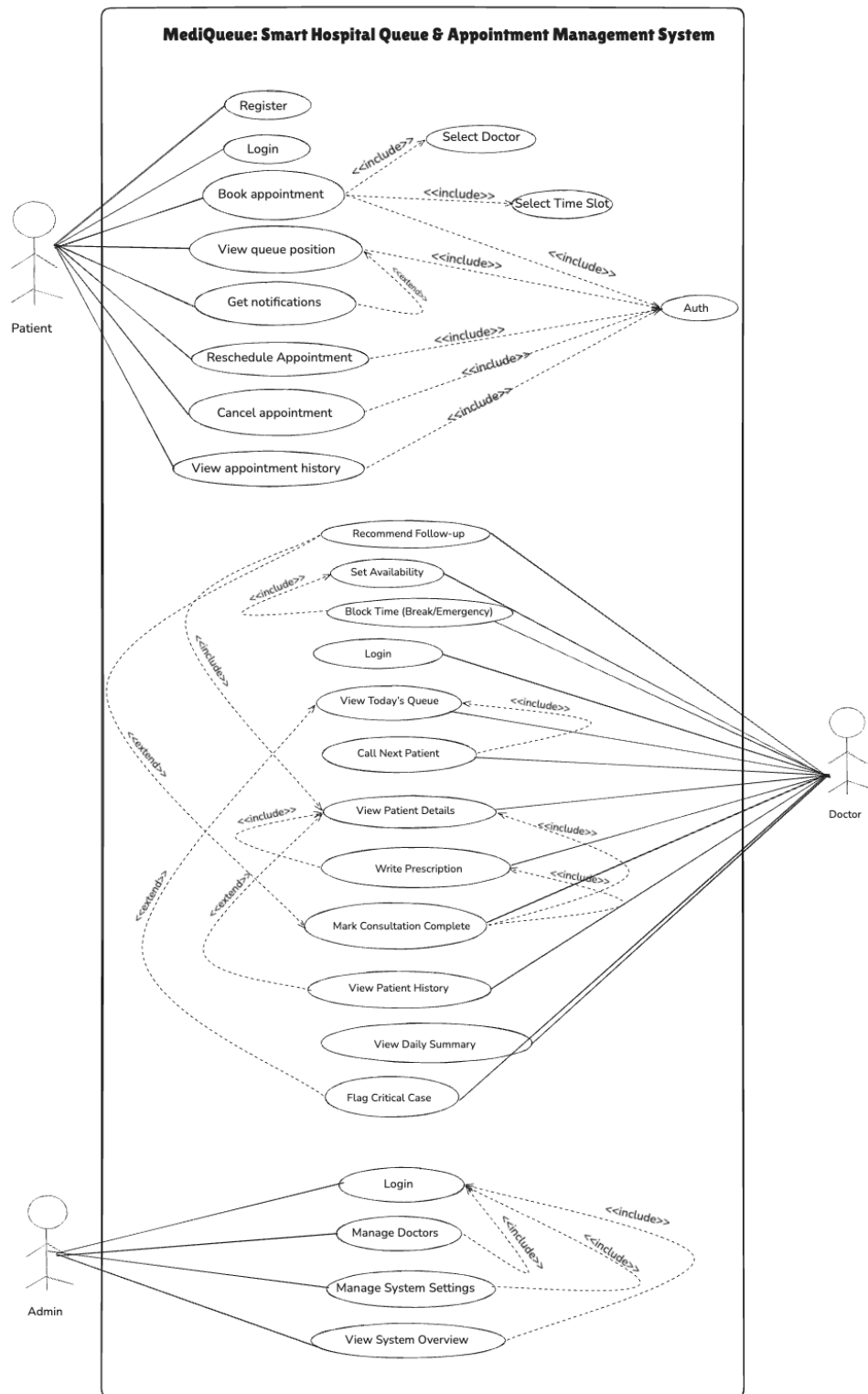


Figure 3. Use case diagram for patient, doctor, and admin interactions.

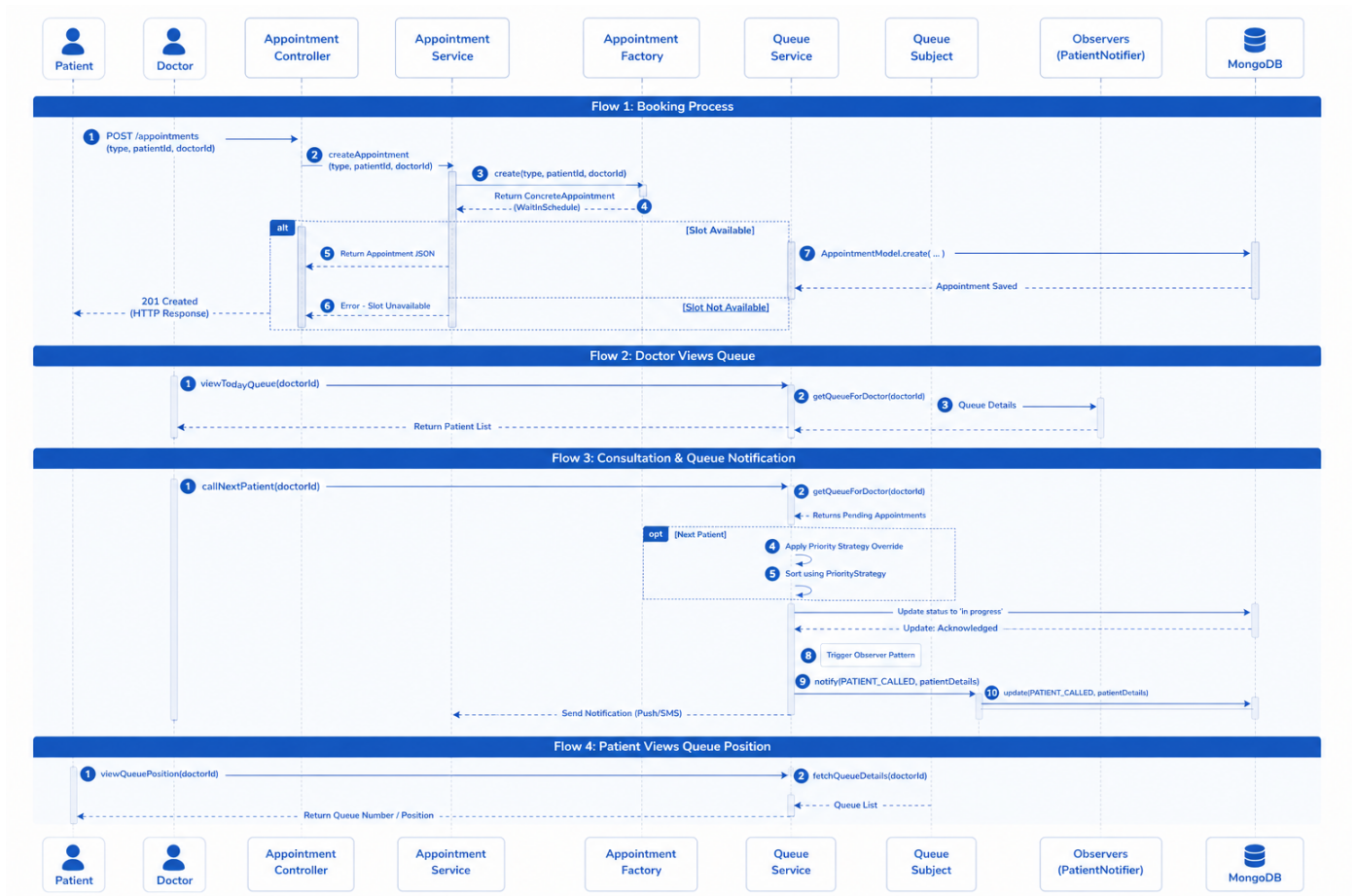


Figure 4. Sequence diagram for a major MediQueue workflow.

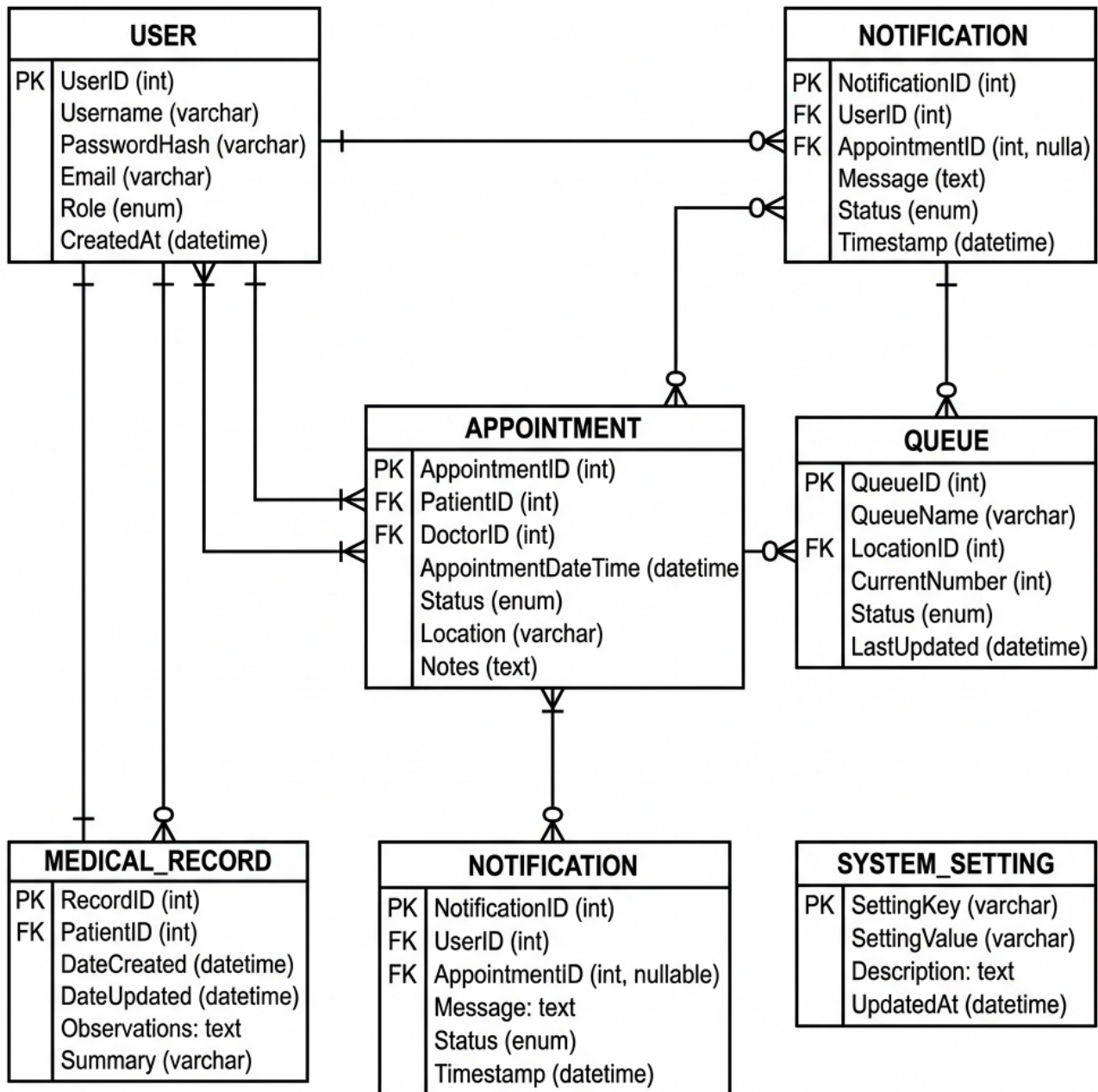


Figure 5. ER diagram for the MediQueue database design.